Unified Training–Learning–Inference–Reinforcement Dynamics Across Neural Systems

Anonymous Authors

Abstract

We study unified Training–Learning–Inference–Reinforcement (TLIR) dynamics across neural systems. Training evolves a parameter–optimizer state: the same realized update can produce different finite-amplitude functional responses in different receiving states. Learning persistently reorganizes distributed functional support, and a target's capability changes when its readout boundary is crossed. Inference recruits and combines learned support at a frozen persistent state; reinforcement couples environmental consequences to subsequent training actions. We establish the connected Training–Learning–Inference (TLI) relations in nanoGPT and test them in ResNet-18, diffusion U-Net, routed Vision-MoE, a graph neural network, and a hard-spiking recurrent network. The experiments include realized-update exchanges, finite-amplitude response paths, component-coalition interventions, target-boundary measurements, and exact trained-version inference rollback. A theory-derived target-boundary predictor classifies immediate outcomes of formed updates across architectures. In a seventh, shared GRU policy, RL-E05/E06 examine how correct-feedback concentration changes functional support, weakens other mastered skills, and enables recovery when feedback is redistributed. The combined results provide an experimentally traceable account of training actions, learning, inference, and reinforcement.

1 Introduction

Training, generalization, capability formation, and inference are central problems in neural-network research. Different fields have developed influential theories to study them.

In *optimization theory*, the Neural Tangent Kernel (NTK) describes parameter-induced output changes through the network Jacobian and the corresponding kernel in specified training regimes [Jacot et al., 2018]. Edge of Stability studies training behavior in relation to the largest Hessian eigenvalue, learning rate, and loss trajectory [Cohen et al., 2021]. Sharpness-Aware Minimization (SAM) constructs training updates using loss changes in a parameter neighborhood to improve generalization [Foret et al., 2021]. Here, the *current state* is represented by parameters and their local derivatives or curvature; *realized updates* are produced by the optimizer; and *responses* appear as changes in model outputs or loss.

In *representation learning and generalization*, Spectral Bias examines frequency-dependent learning rates of functions or kernel modes [Rahaman et al., 2019]. Implicit Bias studies why an optimizer selects particular solutions, such as maximum-margin directions on separable data [Soudry et al., 2018]. Neural Collapse measures class-feature means, within-class variability, and classifier geometry, revealing continuing representational change after training accuracy saturates [Papyan et al., 2020]. Information Bottleneck theory studies mutual information among inputs, hidden representations, and targets, including the retention and compression of task-relevant information [Tishby and Zaslavsky, 2015]. Their *internal organizations* take the form of spectral modes, optimizer-selected solutions, feature geometry, or information structure; their *functional changes* appear in learning rates, evolving solutions, and predictive performance.

In *capability formation*, Grokking captures the long delay between fitting training examples and achieving strong test generalization [Power et al., 2022]. Subsequent circuit studies track the formation and reorganization of internal computations during this delay [Nanda et al., 2023]. Double Descent studies how test error can rise near interpolation and fall again as model size, training time, or data conditions change [Belkin et al., 2019; Nakkiran et al., 2020]. These studies measure the *current state* through training phase or interpolation regime, *internal change* through circuit and solution evolution, and *observable capability* through accuracy and generalization error.

In *continual learning*, catastrophic forgetting occurs when training on a new task reduces performance on earlier tasks. Elastic Weight Consolidation (EWC) estimates parameter importance using Fisher information from earlier tasks and penalizes changes to important parameters [Kirkpatrick et al., 2017]. Gradient Episodic Memory (GEM) constrains new-task gradients using their inner products with previous-task gradients [Lopez-Paz and Ranzato, 2017]. Orthogonal Gradient Descent (OGD) projects new gradients into directions that approximately preserve earlier-task outputs [Farajtabar et al., 2020]. In these methods, the *existing state* includes previously learned parameters and function-sensitive direc-

tions; *updates* are constrained new-task actions; *interactions* occur through overlapping gradient or functional subspaces; and *capability changes* are measured as retention, forgetting, or transfer.

In *inference research*, Attention uses queries, keys, and values to compute input-dependent information weights [Vaswani et al., 2017]. In-context learning studies how examples and task information provided in the context affect predictions while model parameters remain fixed [Brown et al., 2020]. Chain-of-Thought prompting studies how intermediate reasoning steps affect performance on complex tasks [Wei et al., 2022]. The *current state* is the trained model; *input conditions* include the query, context, and intermediate computations; *internal computation* varies with attention and activations; and *functional performance* is observed in the resulting predictions.

In *reinforcement and feedback-driven training*, reinforcement learning from human feedback (RLHF) uses preference and reward signals to adjust model behavior [Ouyang et al., 2022]. Self-training creates further training data from a model's predictions [Xie et al., 2020]. Recursive model-collapse studies examine the loss of low-probability modes when successive models are trained on data generated by their predecessors [Shumailov et al., 2024]. Here, *feedback* is supplied through rewards, preferences, or generated-data distributions; *training actions* are the resulting updates; and subsequent *functional change* is observed through behavior, accuracy, and generated distributions.

These studies use different mathematical objects—kernels, Hessians, gradients, class features, mutual information, task scores, attention weights, and reward signals—yet six recurring categories can be identified: *current state, realized update, functional response, internal organization, observable capability, and feedback. The core TLI relations were established experimentally before this cross-field comparison; identifying these recurring variables here is a way to help readers understand the theory, not the process by which it was discovered.*

2 Experimental Systems and Methods

2.1 Neural systems and indexed experiments

The TLI core was established in nanoGPT through TL-E01–E08 and INF-E01, with TL-P01 testing a predictive consequence. Five additional systems retest its connected roles. RL-E05/E06 examine how correct feedback reallocates already learned functional support and changes retained capabilities in a seventh system, a shared GRU policy. Table 1 lists the executed systems and registered component groups.

System (optimizer)	Task / registered components
nanoGPT (Adam)	Cyclic next-token; 4 attention/MLP branches
ResNet-18 (SGD momentum)	CIFAR-100 classification; 4 residual stages
Diffusion U-Net (AdamW)	CIFAR-10 denoising; 4 U-Net paths
Vision-MoE (AdamW)	CIFAR-100 classification; 4 routed experts
GNN (RMSprop)	Graph reachability; 4 message-passing blocks
Spiking SNN (Adagrad)	8×8 digits; 4 spiking blocks
GRU (RL-E05/E06; AdamW)	Four-skill feedback; 4 shared hidden groups

Table 1: Seven neural systems: six TLI chains and a GRU feedback study.

2.2 Training: realized actions and receiving states

At a registered update, the actual model–optimizer state is $Z_t = (\theta_t, \omega_t)$, with realized parameter action $\Delta\theta_t = \theta_{t+1} - \theta_t$. The experiment records the pre-state, minibatch, gradient, optimizer memory, and native post-state. TL-E01/E02 examine capability trajectories and optimizer-pause/clipping controls. TL-E03 transplants an *identical formed update* between historical receiving states and compares responses relative to each unchanged-state branch:

$$R_q(\theta_A, \Delta\theta_B) \quad \text{vs.} \quad R_q(\theta_B, \Delta\theta_B), \tag{1}$$

where $R_q(\theta, \Delta) = m_q(\theta + \Delta) - m_q(\theta)$ for target q. A separate optimizer-memory fork fixes the current parameters and gradient while substituting historical memory:

$$\Delta^{(a,b)} = \mathrm{OptStep}(\theta_t, g_t, \omega_{a,b}). \tag{2}$$

TL-E04 evaluates the *realized* update at finite amplitudes,

$$r_{t,q}(\alpha) = m_q(\theta_t + \alpha\Delta\theta_t) - m_q(\theta_t), \tag{3}$$

with common cross-system grid $\alpha \in \{0, .125, .25, .5, .75, 1\}$. The paths reveal linear, saturation, acceleration, turnback, and sign-reversal regimes. TL-E05 identifies three main pre-response conditioning families: current target-boundary state (F1), target-specific update geometry (F3), and parameter–optimizer receiving state (F5).

2.3 Learning: support formation and readout

TL-E06 applies reversible zero gates to identified components. For active coalition $C \subseteq \mathcal{C}$, let $v_{t,q}(C) = m_q(f^{(C)}_{\theta_t}(x_q))$. Single/pair interventions measure necessity, backup, substitution, interaction, and failure slack; cross-system implementations also evaluate the 16 coalitions of four components and their exact Shapley effects:

$$\phi_{j,q} = \sum_{C \subseteq \mathcal{C}\setminus\{j\}} \frac{|C|!(3-|C|)!}{4!} \left[v_q(C \cup \{j\}) - v_q(C)\right]. \tag{4}$$

TL-E07 compares the intervention-derived support allocation before and after the *same* update,

$$\Delta P_{t,q} = P_q(\theta_t + \Delta\theta_t) - P_q(\theta_t), \tag{5}$$

including support hand-offs and component interactions. TL-E08 aligns identities and target readout boundaries. Classifiers use $m_q = \ell_y - \max_{c \neq y} \ell_c$; diffusion uses a native residual-competition boundary $m_q = \min_k(E_k - E_{\mathrm{true}})$. The pre/post signs give four outcomes: $(1,1)$, $(1,0)$, $(0,0)$, and $(0,1)$ for maintenance, loss, non-acquisition, and recovery/acquisition.

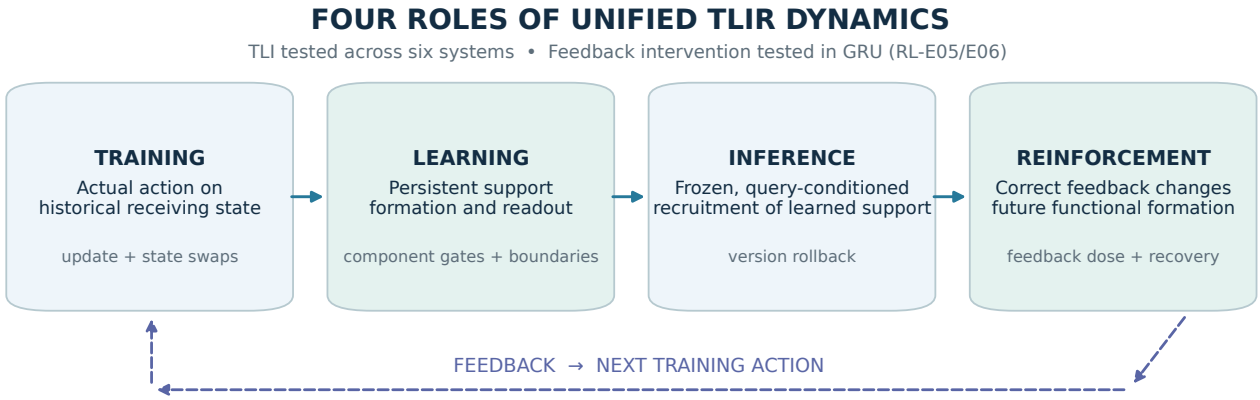


Figure 1: Four peer roles of TLIR. The TLI core is tested in six neural systems; RL-E05/E06 test reinforcement through controlled feedback redistribution in a shared GRU policy.

2.4 Inference: frozen recruitment and rollback

INF-E01 tests 13 nanoGPT histories and 52 identified checkpoints; INF-G01/G02 and the GNN/SNN studies extend these measurements. At a fixed learned state L^*, inference recruits query-conditioned support and combines it in the current occurrence:

$$S_{q,c}^{\text{active}} = \Pi_{q,c}(L^*), \qquad y_{q,c} = H_{q,c}(S_{q,c}^{\text{active}}). \tag{6}$$

Frozen component gates measure non-additivity at the *complete output* level:

$$I_{ij,q} = f_{\text{full}} - f_{-i} - f_{-j} + f_{-i,-j}. \tag{7}$$

Each trained component is separately replaced by its exact pre-learning version, while other trained parameters remain fixed; its output effect is

$$D_{j,q} = f(q;\theta^*) - f(q;\theta^{(j\leftarrow\text{pre})}). \tag{8}$$

The original trained version is then restored, with the output and persistent model/optimizer state checked for exact recovery.

2.5 Reinforcement: correct-feedback redistribution

Reinforcement is a peer TLIR role, not an add-on to TLI. RL-E05 and RL-E06 begin with a shared attention-free GRU policy in which *four capabilities have reached mastery*. All branches receive matched initial parameter–AdamW states. A complete two-decision chain receives $r = 1$ only when correct and $r = 0$ otherwise. The interventions manipulate *where correct feedback is concentrated*, not the reward rule.

RL-E05 compares selective feedback for one mastered capability, balanced feedback for all four, and frozen exposure without parameter updates. All 16 coalitions of four GRU component groups measure Shapley support and task margins; 15 nonempty component-version rollback subsets test the observed capability trade-off.

RL-E06 holds 60 episodes per update and the 3,200-update budget constant while varying feedback allocation (Table 2). At exclusive-feedback update 800, two exact-state recovery forks receive 2,400 further updates: rebalanced $[15, 15, 15, 15]$ or other-skills-only $[0, 20, 20, 20]$. Feedback dose, support share, continuous target margins, correctness, and recovery are measured on the same learned policy. The experiment separates internal margin erosion from later binary capability loss and tests whether changing feedback allocation reverses both.

Feedback	q_0	q_1	q_2	q_3
Balanced	15	15	15	15
Mild	30	10	10	10
High	45	5	5	5
Exclusive	60	0	0	0

Table 2: RL-E06 feedback episodes per update, by capability.

2.6 Prediction and indexed evidence

TL-P01 predicts the immediate target boundary after a *formed* update, before the post-update response is read. For target y, competitor c, hidden output h, $b_c = w_y - w_c$ and $\Delta b_c = \Delta w_y - \Delta w_c$, the frozen `quadratic_complete` algorithm uses pre-update directional derivatives $h' = Dh[\Delta\theta_t]$ and $h'' = D^2h[\Delta\theta_t, \Delta\theta_t]$:

$$\begin{aligned}\widehat{g}_{y,c} = {} & b_c^\top h + \Delta b_c^\top h + b_c^\top h' \\ & + \Delta b_c^\top h' + \tfrac{1}{2} b_c^\top h''.\end{aligned} \tag{9}$$

Correctness is predicted iff $\min_{c\neq y} \widehat{g}_{y,c} > 0$; four target transitions follow from initial and predicted correctness. The prediction is sealed before post-update adjudication. Cross-system predictors adapt directional response computation to each native task.

Records retain concrete target, update and parameter-version identities, predeclared tests and all observed results, with independent checkers or native re-execution used to verify reported outcomes.

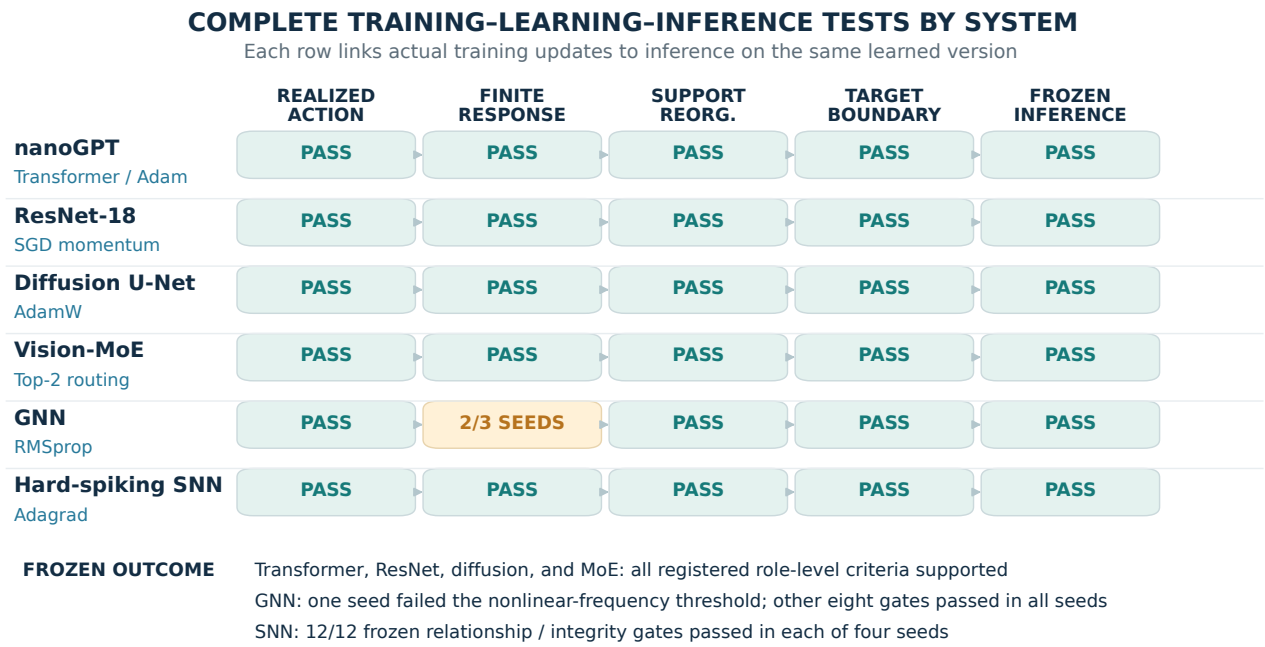


Figure 2: Within-system training–learning–inference experiment chains. One GNN seed meets eight of nine registered gates, with nonlinear-response incidence below its specified threshold; SNN passes all twelve registered gates in four seeds.

Index	Experiment / causal role
TL-E01–E05	Evolution, receiving-state forks, finite paths, response coordinates
TL-E06–E08	Component support, reallocation, capability readout
TL-P01	Post-update target-boundary prediction
INF-E01	Learned-version frozen inference
TL-G01–G03	ResNet, diffusion, MoE: training–learning
INF-G01–G02	ResNet, diffusion, MoE: frozen inference
GNN / SNN	Additional within-system TLI tests
RL-E05 / E06	Selective feedback, interference, dose and recovery

Table 3: Indexed experimental chain and tested relations.

3 Results: Cross-System TLI and Reinforcement

3.1 Native Transformer, ResNet, diffusion and MoE chains

The original nanoGPT program uses realized-update interventions, distributed-support measurements, identity-aligned target transitions, and frozen inference on exact historical trained versions. The frozen inference arm examines thirteen validated histories and 52 selected checkpoints. The post-update target-boundary predictor correctly adjudicates $4{,}652/5{,}088$ confirmation targets (91.43% accuracy, 91.49% four-transition macro recall), with errors retained.

On ResNet, all $42/42$ registered parameter receiving-state exchanges exceed the frozen response-difference threshold; 32.23% of $1{,}536$ target-update response paths meet the nonlinear criterion, and 91.41% of support records exceed the specified reallocation threshold. Its trained checkpoints then enter the frozen-inference protocol, passed by all three seeds, including learned-version rollback and exact restoration.

On the diffusion U-Net, all $18/18$ parameter-state exchanges exceed the threshold. Nonlinear finite responses occur in 19.25% of 504 records, and support reallocation exceeds threshold in 97.02%. The corresponding frozen-inference protocol passes on all three seeds, including intervention effects in the epsilon predictor and complete 100-step deterministic sampling. This is a generative task with a residual-error capability boundary, rather than another class-label classifier.

The Vision-MoE repeats the connected experiment in a routed-expert organization. All $36/36$ parameter-state and $18/18$ Adam-memory exchanges exceed their registered thresholds; nonlinear paths occur in 35.71% of $1{,}344$ target-update records, and expert-support reallocation exceeds threshold in 80.43%. The same final learned versions are tested for deterministic top-2 routing, query-specific expert use, non-additive coalitions, trained expert/router rollback and restoration. The three-seed joint frozen verdict is `MOE_UNIFIED_DYNAMICS_SUPPORTED`.

These four systems therefore supply *repeated, within-system* evidence across the action, finite response, support, boundary, and frozen-inference roles. In particular, the inference evidence is linked to training by exact parameter-version identity, not merely by similar behavior in another model.

3.2 GNN and hard-spiking SNN stress tests

The GNN experiment performs the same realized-update/receiving-state, finite-path, pre/post coalition-support, target-boundary, and exact learned-version inference sequence. All three formal seeds form the intended capability and pass eight frozen criteria. Two seeds pass the ninth, a $\geq 10\%$ nonlinear-response frequency criterion; one has only $4/256$ qualifying paths (1.56%) and *fails*. The state-response, support, capability, and frozen-inference measurements were completed across all three seeds.

The hard-spiking SNN uses four recurrent spike components and six temporal steps. Across four formal seeds, all twelve frozen gates pass, including native Adagrad-memory intervention, finite paths, component-support reorganization, query-conditioned inference and exact version rollback. Its discontinuous hard-threshold forward is particularly informative: the connected functional roles remain observable even where a local derivative-based predictor has difficulty modeling threshold crossings.

3.3 Reinforcement: feedback-directed support and capability change

RL-E05/E06 examine the fourth, equally central TLIR role using a shared four-skill GRU policy. All four skills are mastered *before* the matched feedback branches begin; all positive rewards follow the exact two-action task rule. Thus the intervention changes the *distribution of correct feedback*, not reward validity. RL-E05 establishes selective-feedback support concentration, cross-skill interference, and version-sensitive trade-offs, although its stronger registered large-event timing gate was not met.

In RL-E06, 12 formal seeds exhibit strictly dose-ordered changes: from balanced to mild, high, and exclusive feedback, the reinforced skill's mean relative support shares are 25.54%, 26.47%, 27.50%, and 35.20%, while the other skills' mean margins decrease from 9.474 to 9.130, 8.493, and 3.962. With mild or high concentration, the other skills still achieve 100% accuracy even though their margins have already eroded. Compared with balanced feedback, exclusive correct feedback shifts support share by +9.67 percentage points and decreases other-skill accuracy by 39.06 points. Rebalancing from the *same* exclusive-update-800 state restores other-skill accuracy from 70.31% to 99.48%, while the reinforced skill remains at 100%. The experiments therefore connect feedback allocation to persistent support reorganization, continuous boundary erosion, observable capability loss, and reversible recovery.

3.4 Boundary prediction probes both strength and limits

Direct endpoint prediction measures the accuracy of mathematical approximations alongside the causal intervention tests. Table 4 compares recorded experiments. The primary nanoGPT `quadratic_complete` method is adapted to the output and activation structure of each subsequent system.

For the MoE's direct second-order endpoint predictor, each of the two capability-changing categories contains only nine cases; their high recalls must be interpreted with this limited denominator. On the SNN, the unchanging-boundary baseline already reaches 93.63% accuracy, and the local formula reaches 95.31% overall, but only 70.74% four-way macro recall. Its hard threshold makes true local hidden-state derivatives uninformative about some finite threshold crossings. The lower macro recall isolates a prediction challenge at hard activation thresholds, even as the experimental functional chain remains observable.

System	Accuracy (%)	Four-way recall (%)
nanoGPT	91.43	91.49
ResNet-18	99.28	97.99
Diffusion U-Net	99.60	97.98
Vision-MoE	99.63	96.95
GNN[a]	98.54	98.62
Hard-spiking SNN[a]	95.31	70.74

[a] Independent prediction tests use registered GNN/SNN training histories.

Table 4: Post-update target-boundary classification reports. Macro recall averages four observed transition categories; high accuracy alone is not sufficient. Predictor details and evaluation sets differ by system.

4 Discussion

TLIR is the complete theoretical account, with Training, Learning, Inference, and Reinforcement as four peer roles. Its TLI relations recur within six architecturally different systems, where exact training histories link realized updates, state-dependent responses, persistent support reorganization, capability readouts, and learned-version frozen inference. The seventh system, a shared GRU policy, probes how Reinforcement changes the future of *already formed* capabilities. This combination tests both the formation/use of support and the feedback that subsequently reorganizes it.

The GRU results reveal why reinforcement can be effective and damaging through the *same functional mechanism*: repeated correct feedback preferentially reallocates support toward the rewarded capability, while unreinforced targets lose continuous readout margin and may later cross into failure. Rebalancing feedback from the same learned state largely reverses this process. Reward correctness alone therefore does not determine whether all previously acquired capabilities are preserved. The evidence concerns controllable support reorganization, not a conserved quantity of support.

Quantitative response patterns remain system-specific: nonlinear-response incidence varies, one GNN seed misses its registered frequency gate, and hard-spiking thresholds challenge local derivative-based prediction. The causal TLI tests and the GRU feedback interventions examine complementary parts of TLIR dynamics; their individual empirical scope should not be confused with requiring every system to reproduce every intervention. Component-level measurements also connect to causal-abstraction analyses of mechanisms [Geiger et al., 2025].

5 Unified Explanations of Positive and Negative Findings

Question and discovery order. A separate comparison of 21 historical positive/negative pairs asks whether common receiver–action–response–support–boundary–feedback relations explain both the success of local accounts and the findings that limit their scope. The examples below illustrate this shared interpretation.

Kernel dynamics versus feature learning. In the corresponding infinite-width lazy regime, NTK dynamics give a rigorous local training description [Jacot et al., 2018]; feature-learning parameterizations admit different limiting behavior [Yang and Hu, 2021]. TLI locates both within receiver- and parameterization-dependent action–response regimes: a valid frozen-kernel projection does not imply that all finite-amplitude training trajectories preserve their features. The two results occupy distinct training regimes.

Sharp minima versus useful sharpness-aware training. Function-preserving reparameterizations can change raw parameter sharpness without changing generalization [Dinh et al., 2017]. Nevertheless, SAM improves performance in specified tasks by optimizing a neighborhood-robust loss [Foret et al., 2021]. The paired explanation is that raw parameter geometry is not the complete learned functional state, while a sharpness-aware intervention can still change the *actual training action* beneficially. Thus a non-invariant explanatory scalar and an effective local control method can coexist.

Compression as a useful mechanism, not a universal requirement. General compression-stage claims fail in some architectures and measurement regimes [Saxe et al., 2018], whereas appropriately controlling information bottlenecks can yield valid generalization guarantees [Kawaguchi et al., 2023]. Persistent functional reorganization in TLI does not require a monotonic decrease of $I(X;T)$, but compression can be one regime-specific outcome of the receiving state, noise, and objective. This predicts *bounded utility without universal necessity*.

Delayed emergence and continued learning after mastery. In grokking, test capability can appear abruptly after training performance has saturated, although internal computational structure evolves in identifiable stages [Nanda et al., 2023]. Neural-collapse results likewise document continuing feature geometry after training error vanishes [Papyan et al., 2020]. TLI separates persistent support formation from the discrete target readout boundary. Continued internal formation and sudden observable transitions are thus compatible outcomes of the same measured distinction.

Forgetting versus preservation and recovery. Continual learning can damage older capabilities, while methods such as EWC protect selected high-importance parameter directions [Kirkpatrick et al., 2017]. In TLI, the signed response of an old target depends on overlap between the new action and its function-sensitive support, relative to remaining boundary slack. The GRU feedback forks supply a matched intervention example: concentrated *correct* feedback erodes other mastered skills, whereas rebalanced feedback largely restores them.

Beneficial self-training versus recursive collapse. Pseudo-label-based self-training improves accuracy in suitable data mixtures [Xie et al., 2020]; recursively replacing original data with model-generated samples can instead erase minority tails [Shumailov et al., 2024], while accumulating real data can prevent that degradation in studied regimes [Gerstgrasser et al., 2024]. Both follow a feedback-data distinction: an inference-generated sample is neither intrinsically beneficial nor intrinsically harmful; its retained information and subsequent training action matter. For an under-retained mode with retention factor $0 < \rho < 1$, a conditional illustrative recursion is

$$q_{t+1} = (1-\alpha)\rho q_t + \alpha q_{\mathrm{real}}. \tag{10}$$

At $\alpha = 0$ the mode contracts, whereas $\alpha > 0$ supplies a positive fixed-point floor if $q_{\mathrm{real}} > 0$. The recursion models the case of a repeatedly under-retained data mode.

Theory-first extrapolation. Beyond historical comparisons, the TLIR relations have been used to derive conditional predictions and their test conditions before searching the relevant literature. Examples include post-capability max-margin selection under exponential-tail losses [Soudry et al., 2018] and evolving-state stability at the Edge of Stability [Cohen et al., 2021]. These comparisons identify specific state, action, and boundary conditions under which local mathematical results arise. Future tests can directly intervene on these conditions and evaluate the predicted transitions.

6 Conclusion

We investigate the unified Training–Learning–Inference–Reinforcement (TLIR) dynamics of neural systems. Six different architectures repeatedly exhibit the connected TLI relation among realized actions, receiving-state responses, persistent support reorganization, capability boundaries, and exact-version frozen inference. In a seventh, GRU-based system, RL-E05/E06 show that *correct* feedback can selectively concentrate functional support, weaken other mastered capabilities, and reverse these changes when feedback is redistributed. The predictive boundary algorithm and the unified explanations of previously separate positive and negative findings follow from this experimentally grounded TLIR account.

References

[Belkin et al., 2019] M. Belkin, D. Hsu, S. Ma, and S. Mandal, "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off," *PNAS*, vol. 116, no. 32, pp. 15849–15854, 2019, doi: 10.1073/pnas.1903070116.

[Brown et al., 2020] T. B. Brown et al., "Language Models Are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, 2020.

[Cohen et al., 2021] J. M. Cohen et al., "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability," in *Proc. ICLR*, 2021.

[Dinh et al., 2017] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio, "Sharp Minima Can Generalize For Deep Nets," in *Proc. ICML*, pp. 1019–1028, 2017.

[Farajtabar et al., 2020] M. Farajtabar, N. Azizan, A. Mott, and A. Li, "Orthogonal Gradient Descent for Continual Learning," in *Proc. AISTATS*, pp. 3762–3773, 2020.

[Foret et al., 2021] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-Aware Minimization for Efficiently Improving Generalization," in *Proc. ICLR*, 2021.

[Geiger et al., 2025] A. Geiger et al., "Causal Abstraction: A Theoretical Foundation for Mechanistic Interpretability," *J. Mach. Learn. Res.*, vol. 26, no. 83, pp. 1–64, 2025.

[Gerstgrasser et al., 2024] M. Gerstgrasser et al., "Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data," in *ICML Workshop on Foundation Models in the Wild*, 2024.

[Jacot et al., 2018] A. Jacot, F. Gabriel, and C. Hongler, "Neural Tangent Kernel: Convergence and Generalization in Neural Networks," in *Advances in Neural Information Processing Systems*, 2018.

[Kawaguchi et al., 2023] K. Kawaguchi et al., "How Does Information Bottleneck Help Deep Learning?," in *Proc. ICML*, 2023.

[Kirkpatrick et al., 2017] J. Kirkpatrick et al., "Overcoming Catastrophic Forgetting in Neural Networks," *PNAS*, vol. 114, no. 13, pp. 3521–3526, 2017, doi: 10.1073/pnas.1611835114.

[Lopez-Paz and Ranzato, 2017] D. Lopez-Paz and M. A. Ranzato, "Gradient Episodic Memory for Continual Learning," in *Advances in Neural Information Processing Systems*, pp. 6467–6476, 2017.

[Nakkiran et al., 2020] P. Nakkiran et al., "Deep Double Descent: Where Bigger Models and More Data Hurt," in *Proc. ICLR*, 2020.

[Nanda et al., 2023] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "Progress Measures for Grokking via Mechanistic Interpretability," in *International Conference on Learning Representations*, 2023.

[Ouyang et al., 2022] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems*, 2022.

[Papyan et al., 2020] V. Papyan, X. Y. Han, and D. L. Donoho, "Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training," *PNAS*, vol. 117, no. 40, pp. 24652–24663, 2020, doi: 10.1073/pnas.2015509117.

[Power et al., 2022] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets," arXiv:2201.02177, 2022.

[Rahaman et al., 2019] N. Rahaman et al., "On the Spectral Bias of Neural Networks," in *Proc. ICML*, pp. 5301–5310, 2019.

[Saxe et al., 2018] A. M. Saxe et al., "On the Information Bottleneck Theory of Deep Learning," in *Proc. ICLR*, 2018.

[Shumailov et al., 2024] I. Shumailov et al., "AI Models Collapse When Trained on Recursively Generated Data," *Nature*, vol. 631, pp. 755–759, 2024, doi: 10.1038/s41586-024-07566-y.

[Soudry et al., 2018] D. Soudry et al., "The Implicit Bias of Gradient Descent on Separable Data," *J. Mach. Learn. Res.*, vol. 19, no. 70, pp. 1–57, 2018.

[Tishby and Zaslavsky, 2015] N. Tishby and N. Zaslavsky, "Deep Learning and the Information Bottleneck Principle," in *IEEE Information Theory Workshop*, 2015, arXiv:1503.02406.

[Vaswani et al., 2017] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.

[Wei et al., 2022] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems*, 2022.

[Xie et al., 2020] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-Training With Noisy Student Improves ImageNet Classification," in *Proc. CVPR*, pp. 10687–10698, 2020.

[Yang and Hu, 2021] G. Yang and E. J. Hu, "Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks," in *Proc. ICML*, pp. 11727–11737, 2021.